`vDef` user manual

Blaise Bourdin

October 11, 2024

Contents

Introduction

`vDef` is a reference implementation of the Variational Approach to Fracture [FM98, BFM08] and related problems.

This brief manual lists the main command line options for `vDef` with default values are shown between angled brackets `<>`. Note that auto–generated documentation can always be printed using the `-h` and `-verbose 1` flags, possibly combined with `-dryrun`.

`vDef` uses the exodusII file format for its input and output. Geometric entities corresponding to different materials, models or boundary conditions can be encoded using exodus "element blocks" (cell sets in `vDef` lingo) and "node sets" (referred to as vertex sets). Note that exodusII "side sets" are not supported. Instead, `vDef` uses cell sets of co-dimension 1, *i.e.* "bar" elements in 2D and "triangle" elements in 3d.

There are several variants of `vDef` :

- `vDef` focusses on quasi-static fracture.
- `vDefBT` implements muktiple variants of the *backtracking* algorithm.
- `vDefP` and `vDefUpa` implement two different strategies for coupling damage and plasticity. `vDefP` also has some support for work controlled and crack volume-controled evolution.

Chapter 1

Models and algorithms

1.1 Heat transfer

`vDef` can solve diffusive linear steady-state or transient advective-diffusive heat transfer problems. This heat transfer solver can be used as a standalone simulator, or the temperature field can be plugged in the elasticity of gradient damage simulators. In the sequel, we consider the problem of finding the temperature $T(x,t)$ of a body occupying a region Ω of the two or three–dimensional space and satisfying

$$\begin{cases} \rho(x)c_p(x)\left(\dfrac{\partial T(x,t)}{\partial t} - V\cdot\nabla T(x,t)\right) = \nabla\cdot(K(x)\nabla T(x,t)) + \dot{q}_V(x,t) & \text{in } \Omega, \quad (1.1)\\ K\dfrac{\partial T}{\partial n}(x,t) = \dot{q}_S(x,t) + H(T_e - T(x,t)) & \text{on } \partial_n\Omega, \quad (1.2)\\ T(x,t) = T_b(x,t) & \text{on } \partial_d\Omega, \quad (1.3)\\ T(x,0) = T_0. & (1.4) \end{cases}$$

Where $\dot{q}_V$ and $\dot{Q}_S$ denotes respectively the body and boundary heat fluxes, V is an advection vector, T_b is a prescribed boundary temperature, T_e is a constant surrounding temperature, and T_0 is a constant initial temperature. ρ is the material's density, c_p its specific heat, K is the thermal conductivity, a symmetric tensor, and H is the surface thermal conductivity. For steady state problems, time derivatives are ignored and the initial temperature is used as the initial guess of the iterative solver.

1.2 Variational Phase-Field models of fracture

The damage models implemented in `vDef` are regularization of the energy functional devised in the the variational approach to fracture [AT90, AT92, Gia05, SM13]. These problems are formulated as rate independent unilateral minimization problems of a total energy functional

$$\mathcal{E}_\ell(u,\alpha) := \int_\Omega a(\alpha)W(\mathbf{e}(u),T,\mathbf{e}^p)\,dx - \int_\Omega f\cdot u\,dx - \int_{\partial_n\Omega}\tau\cdot u\,ds + \frac{G_c}{4c_w}\int_\Omega \frac{w(\alpha)}{\ell} + \ell|\nabla\alpha|^2\,dx, \quad (1.5)$$

Symbol	Name	Unit
T	temperature	K
t	time	s
ρ	density	$\mathrm{kg\,m^{-3}}$
c_p	specific heat capacity	$\mathrm{J\,K^{-1}\,kg^{-1}}$
V	advection velocity (vector)	$\mathrm{m\,s^{-1}}$
K	thermal conducticity (symmetric matrix)	$\mathrm{J\,s^{-1}\,K^{-1}\,m^{-1}}$
$\dot{q}_V$	body heat flux	$\mathrm{J\,s^{-1}\,m^{-3}}$
$\dot{q}_S$	boundary heat flux	$\mathrm{J\,s^{-1}\,m^{-2}}$
H	surface thermal conductivity	$\mathrm{J\,s^{-1}\,K^{-1}\,m^{-2}}$

Table 1.1: Nomenclature for the heat transfer module

where u, and α denote the displacement and damage fields, T, f, τ, and $\mathbf{e}^p$ are given temperature, body force, surface force and plastic deformation, ℓ is the material internal length, W is linear elastic potential:

$$W(\mathbf{e}, T, \mathbf{e}^p) := \frac{1}{2}\mathbf{A}\left(\mathbf{e} - T\alpha_L - \mathbf{e}^p\right) : \left(\mathbf{e} - T\alpha_L - \mathbf{e}^p\right).$$

$\mathbf{A}$ is the material's Hooke's law, α_L its linear thermal expansion tensor (a symmetric matrix). The function w is related to the energy dissipation function associated with a distributed damage field and the constant $c_w := \int_0^1 \sqrt{w(s)}\,ds$ is a normalization constant, and G_c the fracture toughness. Three variants are implemented (η being a small *residual stiffness* possibly set to 0):

- "AT1": $a(\alpha) = \eta + (1-\alpha)^2$, $w(\alpha) = \alpha$, and $c_w = 2/3$. Critical stress, $\sigma_c^{1D} = \sqrt{3EG_c/8\ell}$
- "AT2": $a(\alpha) = \eta + (1-\alpha)^2$, $w(\alpha) = \alpha^2$, and $c_w = 1/2$. Critical stress, $\sigma_c^{1D} = (3/16)\sqrt{3EG_c/\ell}$
- "LSk": $a(\alpha) = \eta + \frac{1-w(\alpha)}{1+(k-1)w(\alpha)}$, $w(\alpha) = 1-(1-\alpha)^2$, and $c_w = \pi/4$. Critical stress, $\sigma_c^{1D} = \sqrt{2G_cE/k\pi\ell}$

For "AT1-AT2" comparison of the properties, refer to [PAMM11]. The "LSk" model property is having a linear softening slop controlled by a material constant k. Notice that this model is not convex in α when no elastic energy is involved, but it remains convex if $\mathtt{A}(e(u)-p) : (e(u)-p) \gg G_c/(4cw\ell)$.

In the time discrete formulation, the displacement and damage field at time step t_i are given by

$$(u_i, \alpha_i) = \underset{v\in\mathcal{K}(t_i),\beta\in\mathcal{K}'(\alpha_{i-1},\eta)}{\operatorname{argmin}} \mathcal{E}_\ell(v,\beta), \tag{1.6}$$

where $\mathcal{K}(t_i)$ is the set of kinematically admissible displacement fields at step t_i, and $\mathcal{K}'(\alpha_{i-1},\eta)$ is the set of all damage field satisfying the irreversibility condition

$$\mathcal{K}'(\alpha_{i-1},\eta) = \{\alpha \ : \ 0 \le \alpha(x) \ge \alpha_{i-1}(x) \le 1 \ \forall x \text{ such that } \alpha_{i-1}(x) \ge \eta\}.$$

Note that with these notations, enforcing irreversibility through inequality constraints as proposed in [Gia05, AMMP08, PAMM11] corresponds to $\eta = 0$ whereas equality constraints under a threshold as originally proposed in [BFM00] corresponds to η close to 1.

1.3 Plasticity coupled with damage model

In associated perfect plasticity models, the set of admissible stresses is a closed convex set whose boundary is the yield surface. Classical yield surfaces implemented in `vDef` include

$$\begin{aligned}
\text{Von Mises,} \quad & \sqrt{\frac{3}{2}\sigma_D : \sigma_D} = ||\sigma_D|| \leq \sigma_y \\
\text{Tresca,} \quad & \max_{i\neq j}\{\sigma_i - \sigma_j\} \leq \sigma_y, \quad \sigma_i \text{ are principal stresses} \\
\text{Drucker-Prager,} \quad & ||\sigma_D|| + k\,\mathrm{tr}(\sigma) \leq \sigma_y \\
\text{Mohr Coulomb,} \quad & \frac{\max\{\sigma_i\}}{\sigma_T} - \frac{\min\{\sigma_j\}}{\sigma_C} - 1 \leq 0, \quad \sigma_i \text{ are principal stresses} \\
\text{Cap-Model,} \quad & C_{D2}||\sigma_D||^2 + C_D||\sigma_D|| + C_T\mathrm{tr}(\sigma) \leq \sigma_y
\end{aligned} \tag{1.7}$$

Remarks: Von-Mises and Drucker-Prager are special cases of the Cap Model, and the Tresca model is a special case of the Mohr-Coulomb model.

Appendix A.1 gives a short description of the construction and implementation of perfect plasticity in variational form. This approach can be extended to couple plasticity and damage model presented previously as described in [Ale13, AMMV17, Tan17]. Let $b(\alpha)$ be a given function representing how the elastic domain evolves with the damage variable. The total energy functional accounting for plastic dissipation is

$$\mathcal{E}(u,\alpha,p) = \int_\Omega \left[\frac{1}{2}a(\alpha)\mathtt{A}(e(u)-p):(e(u)-p) + b(\alpha)\int_0^t \sup_{\varsigma\in K}\{\varsigma:\dot{p}\}\mathrm{d}\tau + \frac{k}{4c_w}\left(\frac{w(\alpha)}{\ell} + \ell|\nabla\alpha|^2\right)\right] x \tag{1.8}$$

Two solution strategies are implemented: In `vDefP` , minimization with respect to α and joint minimization in the (u,p) are alternated until convergence (see Algorithm 1). In `vDefUpa` , successive minimizations with respect to u, p, and α are performed until convergence.

Plasticity options are set in the Plasticity and material properties of each cell set

1.4 Unilateral-Contact

The phase-field energy (1.5) does not distinguish traction from compression, so that compressive interpenetrating cracks are possible. Dealing with interpenetration or unilateral contact in variational phase-field models of fracture is still challenging. `vDef` implements several approaches. For a good survey of models of unilateral contact, refer to [Li16].

Unilateral contact options are set in the `UnilateralContact` section of each cell set

1.4.1 Amor-Marigo-Maurini model (Hydrostatic deviatoric)

In this approach, originally introduced in [AMM09], the elastic energy is modified in such a way that only deviatoric and positive hydrostatic strains are allowed along crack surfaces:

$$W^{free}(u,\alpha) = \frac{K}{2}\,\mathrm{tr}^-(e(u)) + a(\alpha)\Big(\frac{K}{2}\,\mathrm{tr}^+(e(u)) + \mu e_D(u):e_D(u)\Big)$$

where $K = \frac{(\lambda+2\mu)}{3}$ in 3D and $K = \frac{(\lambda+2\mu)}{2}$ in 2D. The positive trace is:

Algorithm 1 Damage coupled with plasticity algorithm

Set $\alpha_0 = 0, p_0 = 0$.
for $i = 0$ to N **do**
  while $\left|\alpha_i^{j+1} - \alpha_i^j\right|_{L^\infty} \leq \delta_\alpha$ **do**
    $\alpha_i^{j+1} \longleftarrow \underset{\alpha_{i-1} \leq \alpha \leq 1}{\operatorname{argmin}} \ \mathcal{E}(u_i^j, \alpha, p_i^j)$
    while $\left|p_i^{j+1,k} - p_i^{j+1,k-1}\right|_{L^\infty} \leq \delta_p$ **do**
      $u_i^{j+1,k+1} \longleftarrow \underset{u \in \mathcal{K}_A}{\operatorname{argmin}} \ \frac{1}{2} a(\alpha_i^{j+1}) \mathtt{A}(e(u) - p^{j+1,k}) : (e(u) - p^{j+1,k})$
      $p_i^{j+1,k+1} \longleftarrow \underset{a(\alpha_i^{j+1})\mathtt{A}(e(u_i^{j+1,k+1}) - p_{i-1}) \in b(\alpha_i^{j+1})K}{\operatorname{argmin}} \ \mathtt{A}(p - p_{i-1}) : (p - p_{i-1})$
      $k \longleftarrow k + 1$
    end while
    $p_i^{j+1} \longleftarrow p_i^{j+1,k}$
    $u_i^{j+1} \longleftarrow u_i^{j+1,k}$
    $j \longleftarrow j + 1$
  end while
  $p_i \longleftarrow p_i^j$
  $u_i \longleftarrow u_i^j$
  $\alpha_i \longleftarrow \alpha_i^j$
end for

Algorithm 2 Damage coupled with plasticity algorithm

Set $\alpha_0 = 0, p_0 = 0$.
for $i = 0$ to N **do**
  while $\left|\alpha_i^{j+1} - \alpha_i^j\right|_{L^\infty} \leq \delta_\alpha$ and $\left|p_i^{j+1} - p_i^j\right|_{L^\infty} \leq \delta_p$ **do**
    $u_i^{j+1} \longleftarrow \underset{u \in \mathcal{K}_A}{\operatorname{argmin}} \ \frac{1}{2} a(\alpha_i^{j+1}) \mathtt{A}(e(u) - p^j) : (e(u) - p^j)$
    $\alpha_i^{j+1} \longleftarrow \underset{\alpha_{i-1} \leq \alpha \leq 1}{\operatorname{argmin}} \ \mathcal{E}(u_i^{j+1}, \alpha, p_i^j)$
    $p_i^{j+1} \longleftarrow \underset{a(\alpha_i^{j+1})\mathtt{A}(e(u_i^{j+1}) - p_{i-1}) \in b(\alpha_i^{j+1})K}{\operatorname{argmin}} \ \mathtt{A}(p - p_{i-1}) : (p - p_{i-1})$
    $j \longleftarrow j + 1$
  end while
  $p_i \longleftarrow p_i^{j+1}$
  $u_i \longleftarrow u_i^{j+1}$
  $\alpha_i \longleftarrow \alpha_i^{j+1}$
end for

$$\mathrm{tr}^+ e(u) = \begin{cases} \mathrm{tr}\,(e(u)) & \text{if } \mathrm{tr}\,(e(u)) \geq 0 \\ 0 & \text{else} \end{cases} \tag{1.9}$$

and $\mathrm{tr}^-(e(u)) = \mathrm{tr}(e(u)) - \mathrm{tr}^+(e(u))$

This is the best understood approach, and was recently shown to converge to a variational model of fracture with a constrain on interpenetration (see [CCF18]).

1.5 Volume controlled evolutions

Pressure forces along fracture faces are critical to hydraulic fracturing applications.

The total aperture of the cracks is

$$V = \int_\Gamma [\![u]\!].n \, \mathrm{d}\mathcal{H}^{n-1},$$

and can be approximated by

$$V_\ell(\alpha, u) = \int_\Omega \alpha \,\mathrm{div}\,(u)\mathrm{d}x = -\int_\Omega \nabla u.\nabla\alpha + \int_{\partial\Omega} \frac{\partial\alpha}{\partial n} u \mathrm{d}s.$$

For sake of simplicity let $\frac{\partial\alpha}{\partial n} = 0$ on $\partial\Omega$ and assume that a constant pressure is applied on the crack.

Accounting for the work of the pressure force along the cracks

$$W^{crackpresforce} = pV = -p\int_\Omega \nabla u.\nabla\alpha \mathrm{d}x,$$

the total energy (1.5)becomes

$$\mathcal{E}_\ell(u,\alpha) := \int_\Omega a(\alpha) W(\mathbf{e}(u))\, dx - \int_\Omega f\cdot u\, dx - \int_{\partial_n\Omega} \tau\cdot u\, ds + \frac{G_c}{4c_w}\int_\Omega \frac{w(\alpha)}{\ell} + \ell|\nabla\alpha|^2\, dx + p\int_\Omega \nabla u.\nabla\alpha \mathrm{d}x, \tag{1.10}$$

In general, crack propagation driven by the crack pressure p is unstable in the sense of Griffith. Instead, it is common to consider an *injected volume driven* evolution, assuming that all cracks are filled with an incompressible fluid at a constant pressure p Given a target volume V and for a fixed α, the equilibrium pressure can be evaluated by solving for mechanical equilibrium for an arbitrary crack pressure $\bar{p}$. Let $\bar{V}$ be the fracture volume associated to $\bar{p}$. then, the pressure required to achieve a total fracture aperture is

$$p_t = \frac{V_t \bar{p}}{\bar{V}}.$$

1.6 Backtracking algorithms

Backtracking algorithms are based on enforcing necessary conditions for optimality for the entire time evolution [Bou07]. They are based on a comparison principle: at each step, a test field

admissible for all previous times is built and its energy compared to that of the previously computed ones. If the test field is of lesser energy that the previously computed one, the algorithm backtracks to this step and the alternate minimizations are restarted from the newly constructed test field. A standard and a *deep* backtracking are implemented:

Algorithm 3 Standard backtracking algorithm

1: Given N, $t_0 < t_1 < \cdots < t_N$, BT_{tol}:
2: $i \leftarrow 0$
3: **repeat**
4: Compute (u_i, α_i) using alternate minimizations
5: **for** $j = i - \mathrm{BT}_{scope}$ to $i - 1$ **do**
6: **if** $\frac{{t_j}^2}{{t_i}^2}\mathcal{U}_{t_j}(u_{t_i}, \Gamma_{t_i}) + \mathcal{S}(\Gamma_t) \leq \mathcal{U}_{t_j}(u_{t_j}, \Gamma_{t_j}) + \mathcal{S}(\Gamma_{t_j}) - \mathrm{BT}_{tol}$ **then**
7: $i \leftarrow j$
8: goto 4
9: **end if**
10: **end for**
11: **until** $i = n$
12: $i \leftarrow i + 1$

In the deep backtracking, the exploration loop (lines 5-10) is moved into the alternate minimizations loop and executed every `BT_Iinterval` iterations. In addition, the exploration loop can be run forward (`BT_Type Forward`) or backward (`BT_Type Backward`).

1.7 General limitations.

- `vDef` is based on PETSc-3.3 [BGCMS97, BB+12, BBB+12]. Unstructured meshes are handled using `Sieve` [KK09]. A new version based on PETSc' `DMplex` is under development. PETSc-specific options are not described in this manual.
- `vDef` only supports the exodusII file format. Cubit, or Trelis can generate such files. VisIt, ParaView, or EnSight can open such files.
- `vDef` can handle linear and quadratic simplicial finite elements. Due to limitations of the exodus file formal, mixing element types is not supported at this point.
- `vDef` can import element blocks of co-dimension 0 and 1, and node sets. *Node sets may not overlap.*
- Boundary conditions can be specified on cell or vertex sets. Passing boundary conditions on vertex sets is slightly more efficient, but ensuring that vertex sets are disjoint can be difficult.
- Boundary condition handling is *additive*. If a cell or vertex belong to several sets, the type of boundary conditions is given by performing a logical `OR` on the boundary conditions type of its parent entities.
- Element blocks but be numbered in order of increasing co-dimension (*i.e.* surface elements before line elemets in 2D, volume elements before surface elements in 3D). Failure to do so

will lead the following PETSc error message:
`[0]PETSC ERROR: Invalid argument: Input array to small for restrictClosure()!.`

Chapter 2

Command line options and YAML configuration file

2.1 Options strings vs. YAML files

Command line options can also be passed to vDef through an option file, using the `-options_file` command line flag. Machine readable YAML files can be used with the flag `-options_file_yaml`. YAML parsing is fragile, incomplete, and somewhat experimental (alias and links are not implemented, and error reporting is lacking), but much easier to parse. PETSc uses underscores as field delimiters in its options handling and keeps track of hierarchy of options through prefixes, Whereas YAML relies on indented lists. The correspondence between standard options and YAML file straightforward, with a few caveats: array arguments must be given as space delimited in PETSc options but comma delimited without spaces in YAML and boolean arguments can be given the value `0,false,no` or `1,true,yes` in PETSc but only `0,no` or `1,yes` are acceptable in a YAML option file. An example of translation from PETSC–like to YAML options is given below:.

Command line options:

```
-time_min 0 -time_max 1 -time_interpolation linear -time_numstep 11  -dryrun \
-cs0001_hookeslaw 1. 0. 0. .5 0. 1. -cs0001_fractureToughness .1 \
-cs0001_internalLength 01 -cs0001_residualStiffness 0.\
-cs0001_defectLaw_type gradientDamage \
-cs0001_defectLaw_gradientDamage_type AT1 \
-cs0001_displacementBC true false false -cs0001_boundaryDisplacement 1. 0. 0. \
-cs0001_damageBC false
```

Equivalent YAML file:

```
time:
    min: 0
    max: 1
    interpolation: linear
    numstep: 11
```

```
cs0001:
    HookesLaw: 1.,0.,0.,.5,0.,1.
    fractureToughness: .1
    internalLength: .1
    residualStiffness: 0.
    defectLaw:
        type: GradientDamage
        gradientDamage:
            type: AT1
    displacementBC: yes,no,no
    boundaryDisplacement: 1.,0.,0.
    damageBC: no
```

YAML parsing is fragile. Most errors in the YAML file will lead to a deadloop in the parser. A simple script `$\MEF90_DIR/Tests/YAMLValidator` can be used to test-parse a yaml file:

```
YAMLValidator -options_file_yaml <yaml file>
```

If the parsing is successsful, the petsc options version of the yaml file will be displayed.

The most common errors leading to an unparseable YAML file are

1. tabulation signs,
2. windows line ending,
3. inconsistent indentation.
4. spaces in list (`boundaryDisplacement: 1., 0.,0.` is parsed as `boundaryDisplacement: 1.` with all entries following the space being ignored)

2.2 General options

vDef can handle input and result files in two different ways: through the `-prefix` option or the `-geometry -result` option pair.

```
-prefix : File name prefix
-verbose <1>: Verbosity: level
-dryrun <FALSE>: Dry run in order to validate the options file.
-file_format <EXOSingle>: (MEF90FileFormat) I/O: file format.
                          Choose one of EXOSingle EXOSplit
```

Notes:

1. If `-prefix <prefix>` is specified, vDef will look for an input mesh is named `<prefix>.gen`, results will be saved as `<prefix>_out.gen` or `<prefix>-\%4i.gen` if output in split files exodus files is chosen), energies will be saved in `<prefix>_out.ener` for the entire domain and `<prefix>_out-\%4i.enerblk` for each cell set.
2. If `-geometry <geom.gen> -result <res.gen>` is given, vDef will look for an input mesh is named `<geom.gen>`, results will be saved as `<result.gen>, energies will be saved in` \verb¡result¿.ener+ for the entire domain and `<result>-\%4i.enerblk` for each cell set.
3. If `-file_format EXOSplit` is selected (not recommended), each core will save its own part of the geometry in a separate file. Continuity at the subdomain interfaces is not preserved (no ghost points or interface matching informations are saved).

2.2.1 Ordering of tensors

In all options, symmetric tensors or order 2 (symmetric matrices) are given in Voigt notations, *i.e.* a 3×3 symmetric matrix is represented by the array $A_{11}, A_{22}, A_{33}, A_{23}, A_{13}, A_{12}$ and a 2×2 symmetric matrix as A_{11}, A_{22}, A_{12}. The symmetric fourth order tensors (Hooke's laws) are stored as the rows of the upper diagonal part of a matrix consistent with the ordering above, i.e.

$$\begin{bmatrix} C_{1111} & C_{1122} & C_{1133} & C_{1123} & C_{1113} & C_{1112} \\ & C_{2222} & C_{2233} & C_{2223} & C_{2213} & C_{2212} \\ & & C_{3333} & C_{3323} & C_{3313} & C_{3312} \\ & & & C_{2323} & C_{2313} & C_{2312} \\ & & & & C_{1313} & C_{1312} \\ & & & & & C_{1313} \end{bmatrix}$$

becomes: $[C_{1111}\ C_{1122}\ C_{1133}\ C_{1123}\ C_{1113}\ C_{1112}\ C_{2222}\ \ldots C_{1312}\ C_{1313}]$.

A small utility in `$MEF90_DIR/Tests/HookeLaws` can be used to compute the coefficients of isotropic Hooke's laws:

```
bourdin@galerkin:Tests $ ./HookeLaws -E 1 -nu .3
 # MEF90: hg changeset 1792:b88429f29d30
 # Copyright (c) 1998-2014 B. Bourdin <bourdin@lsu.edu>
 # PETSC_ARCH=Darwin-gcc5.0-mef90-g
 # PETSC_DIR=/opt/HPC/petsc-3.3-p7

Hooke's laws for E= 1.00E+00 nu= 3.00E-01
 3D isotropic
 1.34615E+00, 5.76923E-01, 5.76923E-01, 0.00000E+00, 0.00000E+00, 0.00000E+00,
 1.34615E+00, 5.76923E-01, 0.00000E+00, 0.00000E+00, 0.00000E+00, 1.34615E+00,
 0.00000E+00, 0.00000E+00, 0.00000E+00, 3.84615E-01, 0.00000E+00, 0.00000E+00,
 3.84615E-01, 0.00000E+00, 3.84615E-01
 2D plane stress
 1.09890E+00, 3.29670E-01, 0.00000E+00, 1.09890E+00, 0.00000E+00, 3.84615E-01
 2D plane strain
 1.34615E+00, 5.76923E-01, 0.00000E+00, 1.34615E+00, 0.00000E+00, 3.84615E-01
```

Alternatively, *isotropic* Hooke's laws are also implemented (see Section 2.3.1).

2.2.2 Time interpolation

```
time:
  interpolation: <linear> #interpolation type (choose one of)  linear Vcycle quadratic exo
  min: <0>       # "time" value at the first step
  max: <1>       # "time" value at the last step
  numstep: <11>: # number of time steps
  skip: <0>      # number of time steps skipped (use for multi stage simulations)
  numCycle: <1>  # number of cycles for cyclic loads
```

Notes:

1. If `-time_interpolation exo` is selected, analysis times will be loaded from the *output* exodus file, which is expected to exist.
2. `-time_interpolation quadratic` corresponds to the natural time scaling in many heat transfer problems, i.e. τ is linearly interpolated between $\sqrt{t_{min}}$ and $\sqrt{t_{max}}$, and $t_i = \tau_i^2$.

2.2.3 Offset and scaling

It is possible to specify the order in which fields in an exodus file are saved by specifying and *offset*. An offset of 0 can be used to indicate that a specific field is not to be saved in the file.

Piecewise constant loadings functions (fluxes, forces, boundary values) can be passed through the command line. Their time dependence can be controlled using the keywords `constant`, `linear`, or `null`. When the scaling is set to null, `vDef` will skip assembly of the field, when possible. The `EXO` keyword is used to indicate that the values are to be read from the *output* file.

2.3 Problem description

Due to a bug in the ExodusII fortran bindings, `vDef` can only import Exodus entities numbers and ignores their names. When passing command line options, Exodus' element blocks are abbreviated as `cs` (for Cell Sets) and nodes sets as `vs` (for Vertex Sets) concatenated with the entity number formatted with 4 digits. For instance, the prefix associated with exodusII element block 10 would be `cs0010_`.

2.3.1 Material properties

Material properties for a cell set are passed by prefixing the following options with the cell set number (i.e. `-cs0001_Density`, for instance). Materials names are not used at the moment. Substitute the `cs0001_` prefix with the appropriate reference.

```
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Name: <MEF90Mathium2D> # unused
  Density: <1> # [kg.m^(-2)] (rho) Density
  FractureToughness: <1> # [N.m^(-1)] (G_c) Fracture toughness
  SpecificHeat: <1> # [J.kg^(-1).K^(-1)] (Cp) Specific heat
  ThermalConductivity: <1 2 3 > # [J.m^(-1).s^(-1).K^(-1)] (K) Thermal conductivity
  LinearThermalExpansion: <1 2 3 > # [K^(-1)] (alpha) Linear thermal expansion matrix
  hookeslaw:
    type: <Isotropic> # Type of Hooke's law (choose one of)  Full Isotropic
    # for isotropic Hooke's laws only
    YoungsModulus: <1> # [N.m^(-2)] (E) Young's Modulus
    PoissonRatio: <0.3> # [unit-less] (nu) Poisson Modulus
    planeStress: <FALSE> # Use plane stress elasticity (2D only)
    # For full Hooke's laws:
    tensor: <> # coefficients of the Hookes law as described in Section
    internalLength: <1> # [m] (l) Internal Length
    CoefficientLinSoft: <0> #[] (k) Linear softening coefficient for LinSoft
    yieldStress: <1> #[N.m^(-2)] (sigma_y) stress threshold for plasticity
```

```
    residualyieldStress: <0> #[unit-less] (eta) residual yield stress
    DuctileCouplingPower: <2> #[] power of the coupling function b(\alpha)
                              # between the damage and the plasticity models
    # Options for cap model plasticity:
    #    CD || dev(stress) || - C2 tr(stress)^2 - C1 tr(stress) - C0 <= 0
    CoefficientCapModel0: <-0.3> # C0
    CoefficientCapModel1: <0.4>  # C1
    CoefficientCapModel2: <1>    # C2
    CoefficientCapModelD: <1>    # CD
    # options for Drucker Prager plasticity:
    #    || dev(stress) || - k tr(stress) - yieldStress <= 0
    CoefficientDruckerPrager: <-0.5> #k
    # options for Hill plasticity:
    CoeffF: <0.3> #[unit-less] (F)
    CoeffG: <0.3> #[unit-less] (G)
    CoeffH: <0.3> #[unit-less] (H)
    CoeffM: <1> #[unit-less] (M)
    CoeffN: <1> #[unit-less] (N)
    CoeffL: <1> #[unit-less] (L)
    YieldTau0: <1> #[N.m^(-2)] (tau_0)
    residualYieldTau0: <0> #[unit-less] residual stress threshold
    phi1: <0> #[radians] Bunge-Euler angle
    phi2: <0> #[radians] Bunge-Euler angle
    Phi: <0> #[radians] Bunge-Euler angle
    # options for Gurson and Green plasticity:
    delta: <0.0001> #[unit-less] residual in the definition of the porosity
    # Options for Winkler-type model (thin film over a substrate, 2D only)
    cohesiveStiffness: <0> #[N.m^(-4)] (k) cohesive stiffness in Winkler-type models
    residualStiffness: <1e-09> #[unit-less] (eta) residual stiffness
    isLinearIsotropicHardening: <FALSE> #[bool] Plasticity with Linear Isotropic Hardening
    isNoPlCoupling: <FALSE> #[bool] Coupling between damage and plastic dissipation
```

2.3.2 Heat transfer options

Global options

```
heatxfer:
  timeStepping:
    type: <SteadyState> # Type of heat transfer computation (choose one of)
                        # null SteadyState Transient
  addNullSpace: <FALSE> # Add null space to SNES
  initialTemp: <0> # [K] (T): Initial Temperature
  boundaryTemp:
    scaling: <linear> # Boundary temperature scaling (choose one of)
                      # constant linear file null
    Offset: <0> # Position of boundary temperature field in EXO file
```

```
  externalTemp:
    scaling: <linear> # External Temperature scaling (choose one of)
                      # constant linear file null
    Offset: <2> # Position of external temperature field in EXO file
  flux:
    scaling: <linear> # Heat flux scaling (choose one of)  constant linear file null
    Offset: # Position of heat flux field in EXO file
```

Cell sets options

```
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Flux: <0> # [J.s^(-1).m^(-3) / J.s^(-1).m^(-2)] (f): Internal / applied heat flux
  SurfaceThermalConductivity: <0> # [J.s^(-2).m^(-1).K^(-1)] (H) Surface Thermal Conductivity
  externalTemp: <0> # Reference temperature T [K]
  TempBC: <FALSE> # Temperature has Dirichlet boundary Condition (Y/N)
  boundaryTemp: <0> # Temperature boundary value
```

Notes:

1. Surrounding temperature is assumed to be constant. This could easily be changed.

Vertex sets options

```
vs0100: # ID of cell set formatted with 4 digits padded to the left by zeros
  TempBC: <FALSE> # Temperature has Dirichlet boundary Condition (Y/N)
  boundaryTemp: <0> # Temperature boundary value
```

2.3.3 Damage and plasticity options

Global options

The global options are in the `Defmech` Section of the YAML file:

```
DefMech:
  TimeStepping:
    Type: <QuasiStatic> # Type of defect mechanics Time steping (choose one of)
                        # Null QuasiStatic
  solver:
    Type: <AltMin>      # Type of defect mechanics solver (choose one of)
                        # AltMin QuasiNewton1 QuasiNewton2
  disp:
    addNullSpace: <TRUE> # handle rigid motion automatically
  displacement:
    Offset: <3> # Position of displacement field in EXO file
  damage:
    Offset: <2> # Position of damage field in EXO file
    atol: <1.e-3> # Absolute tolerance on damage error
```

```
boundaryDamage:
  Offset: <0> # Position of boundary damage field in EXO file
stress:
  Offset: <6> # Position of stress field in EXO file
temperature:
  Offset: <1> # Position of temperature field in EXO file
plasticStrain:
  Offset: <0> # Position of the plastic strain field in EXO file
  atol: <1.e-4> # Absolute tolerance on plastic strain
cumulatedPlasticDissipation:
  Offset: <1> # Position of the Cumulated Plastic Plastic Dissipation field in EXO file
boundaryDisplacement:
  scaling: <linear> # Boundary displacement scaling (choose one of)  constant linear file null
  Offset: <3> # Position of boundary displacement field in EXO file
boundaryDamage:
  scaling: <constant> # Boundary damage scaling (choose one of)  constant linear file null
force:
  scaling: <linear> # Force scaling (choose one of)  constant linear file null
  Offset: <4> # Position of force field in EXO file
pressureForce:
  scaling: <linear> # Pressure force scaling (choose one of)  constant linear file null
  Offset: <3> # Position of pressure force field in EXO file
CrackPressure:
  scaling: <linear> # Crack Pressure scaling (choose one of)  constant linear file null
  Offset: <0> # Position of Crack Pressure field in EXO file
maxit: <1000> # Maximum number of alternate minimizations for damage
pclag: <10> # Interval at which the PC is recomputed during alternate minimization
irrevThres: <0> # Threshold above which irreversibility is enforced (0 for monotonicity, .99 for
SOR:
  Omega:  <1> # Alterate Minimization over relaxation factor (>0 for limited, <0 for projected)
InjectedVolume:
  atol: <1.e-3> # Absolute tolerance on injected volume error
dampingCoefficient:
  displacement: <0> # Damping coefficient on displacement field
                    # (0 for minimization, 1 for semi-implicit gradient flow)
  damage: <0> # Damping coefficient on damage field
              # (0 for minimization, 1 for semi-implicit gradient flow)
BT:
  Type: <Null> # Backtracking type (choose one of)  Null Backward Forward
  Interval: <-1> # Interval at which Backtracking is run in inner loop (0 for outer loop)
  Scope: <-1> # Backtracking scope (0 for unlimited)
  Tol: <0.01>: # Backtracking relative tolerance
```

Notes:

1. `-BT_Interval 1` is for *deep* backtracking, `-BT_Interval -1` for a standard backtracking.

Cell sets options

```
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Force: <0 1 2 > # [N.m^(-3) / N.m^(-2) / N.m^(-1)] (f): body / boundary force
  pressureForce: <0> # [N.m^(-2) / N.m^(-1)] (p): boundary pressureforce
  CrackPressure: <0> # [unit-less] internal crack pressure
  damage:
    type: <AT1> # Type of damage law (choose one of)
                # AT1 AT2 LinSoft AT1Elastic AT2Elastic LinSoftElastic
  plasticity:
    type: <None> # Type of plasticity law (choose one of)
                 # None Tresca VonMises VonMisesPlaneTheory CapModel
                 # DruckerPragerCapModel VonMises1D HillPlaneTheory Green Gurson
  unilateralContact:
    type: <None> # Type of handling of unilateral contact (choose one of)
                 # None HydrostaticDeviatoric Deviatoric
  DisplacementBC: <0 1 2 > # Displacement has Dirichlet boundary Condition (Y/N)
  boundaryDisplacement: <0 1 2 > # [m] (U): Displacement boundary value
  DamageBC: <FALSE> # Damage has Dirichlet boundary Condition (Y/N)
  boundaryDamage: <0> # [unit-less] (alpha): Damage boundary value
  CrackVolumeControlled: <FALSE> # Crack Pressure controlled by the crack volume
                                 # in this block (Y/N)
  WorkControlled: <FALSE> # Force magnitude controlled by its work in this block (Y/N)
```

Note that +damage+, +plasticity+, and +unilateralContact+ have no effect on cell sets of codimension 1 (lines in 2D and surfaces in 3D).

Vertex sets options

```
vs0100: # ID of cell set formatted with 4 digits padded to the left by zeros
  DisplacementBC: <0 1 2 > # Displacement has Dirichlet boundary Condition (Y/N)
  boundaryDisplacement: <0 1 2 > # [m] (U): Displacement boundary value
  DamageBC: <FALSE> # Damage has Dirichlet boundary Condition (Y/N)
  boundaryDamage: <0> # [unit-less] (alpha): Damage boundary value
```

Chapter 3

Examples

3.1 Uniaxial tension

This example is studied in depth in [BFM08, Section 3.1] in the Griffith setting and in [PAMM11] for gradient damage models.

We consider a rectangular domain $\Omega = (-L/2, L/2) \times (-H/2, H/2)$ subject to a monotonically increasing uniaxial displacement or surface force on its lateral edges.

Figure 3.1: Domain for the uniaxial tension example. The numbers correspond to cubit entities.

Trelis journal files for this problem are located in `Examples/UniaxialTension/UniaxialTension2D.jou` and `Examples/UniaxialTension/UniaxialTension3D.jou`. A sample YAML option file is located at `Examples/UniaxialTension/UniaxialTension.yaml`

Proposed numerical experimentations:

1. Using `defectLaw_type AT1` and `defectLaw_type AT1`, study the interaction between mesh size and regularization length. For a given mesh, see how the surface energy depends on ℓ. Check that this dependency can be avoiding by using an effective fracture toughness.
2. Notice that the numerical solution without backtracking does not satisfy energy continuity and cannot be a global minimizer. Use a backtracking algorithm to compute a global minimizer.
3. Without backtracking, see how the loading upon which crack nucleate depends on ℓ.
4. See how force driven computations will fail as soon as a critical force is reached.

5. Compare algorithmic complexity, parallel performances of linear and quadratic finite elements for this problem.

3.2 Double Cantilever Beam

It is well known that crack propagation in a standard shaped double cantilever beam sample is not stable. We consider a tapered geometry as shown in Figure 3.2. Various Trelis journal files for this problem are located in `Examples/DCB`. For $L_1 = 1.25$, $L_2 = 4$, $L_3 = 1$, $R = .25$, $H_1 = 1$, $H_2 = 2$, a mesh size $h = .05$ will give approximately 25,0000 elements. A fracture toughness of $k = .01$ will lead to crack propagation when $0 \leq t \leq 1$.

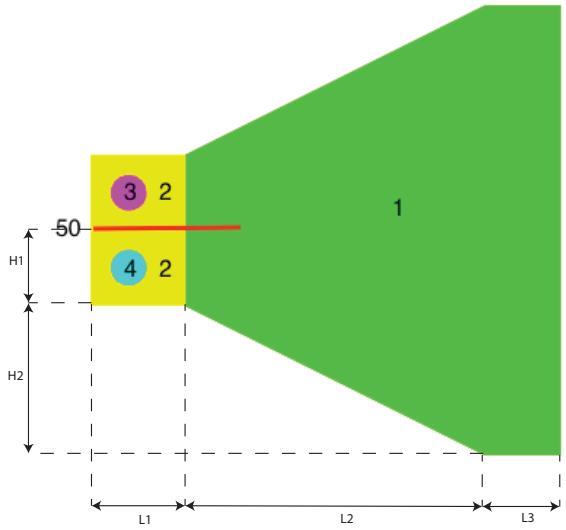

Figure 3.2: Domain for the DCB example. The numbers correspond to cubit entities.

Proposed numerical experimentations:

- Prescribe the y component of the displacement in cell sets 3 and 4. A symmetry argument could be used to infer that propagation will take place along the symmetry axis. Is this the case?
- Fix both components of the displacement field on cell sets 3 and 4.
- Add an inclusion in the crack path. See how varying the toughness and stiffness of the inclusion will either capture or divert the propagating crack.

3.3 Surfing

We consider a rectangular domain occupying a domain $\Omega = [L/2, L/2] \times [-H/2, H2]$ with a pre-existing crack $[-L/2, x_c] \times \{0\}$. A *surfing boundary condition* can be used to estimate effective

fracture properties of heterogeneous materials [HHBB14] or as a verification numerical simulation for a brittle fracture simulator. It is a translating plane-stress Mode-I asymptotic far field boundary displacement is applied on the boundary of the domain. On $\partial\Omega$, and given a rate of loading V, one the boundary displacement $u(x, y, t) = U(x - Vt, y)$, is given by (see [Zeh12])

$$\begin{cases} U_x = \dfrac{K_I}{2\mu}\sqrt{\dfrac{r}{2\pi}}(\kappa - \cos\theta)\cos\dfrac{\theta}{2} \\ U_y = \dfrac{K_I}{2\mu}\sqrt{\dfrac{r}{2\pi}}(\kappa - \cos\theta)\sin\dfrac{\theta}{2}, \end{cases} \tag{3.1}$$

where $\kappa = \frac{3-\nu}{1+\nu}$, $\mu = \frac{E}{2(1+\nu)}$, and (r, θ) are polar coordinates emanating from $(x - Vt, y)$, and $K_I = \sqrt{G_c E}$ is the mode-I stress-intensity factor.

For this loading, it is expected that a crack will start propagating along the x–axis at $t = x_c/V$, and that the rate of growth of the crack is exactly V. This loading cannot be described directly from the command line. A python script in `$MEF90_DIR/bin/vDefSurfingBC.py` can be use to write the proper boundary displacement in an exodusII file. A sample YAML option file where the entity numbering is that of Figure 3.3 is as follow:

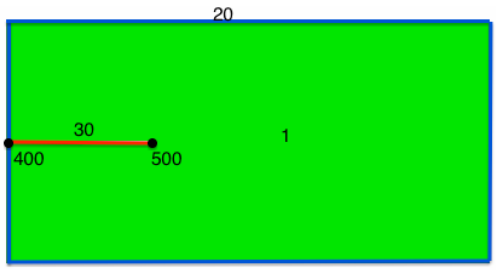

Figure 3.3: Domain for the Surfing example. The numbers correspond to cubit entities.

Proposed numerical experimentations:

1. Check the qualitative fracture behavior in a surfing experiment (crack path, initial length at reactivation, ...).
2. Study the effect of the damage boundary condition along the crack or at the crack tip on the crack propagation. Is activation always progressive?
3. Study the effect of the numerical effective toughness on crack loading at initiation. Can this effect be accounted for *a priori*?

3.4 Enforcing Displacement boundary conditions

Consider the domain from figure 3.4. Prescribe the vertical component of the displacement along the left edge, while the lower edges are clamped. See how if the damage is left free on the lower edge, the surface energy of a boundary crack is mis-calculated.

Figure 3.4: Domain for the Surfing example. The numbers correspond to cubit entities.

3.5 Cooling

This problem replicates the thermal shock experiment studied in depth in [SM13, BMMS14]. A rectangular domain held at initial temperature T_0 is subject to the boundary temperature $T(x,t) = T_0 - \Delta T$ on its boundary. In two dimension, the behavior depends on a single parameter quantifying the intensity of the thermal shock in relation with the material thermo-mechanical properties:

3.5.1 Non-dimensional form of the energy

Assuming that $T0 = \Delta T$ and neglecting plastic deformation, the fracture energy functional (1.5) is

$$\mathcal{E}_\ell(u,\alpha,T) := \int_\Omega a(\alpha) W(\mathbf{e}(u),T)\,dx + \frac{G_c}{4c_w}\int_\Omega \frac{w(\alpha)}{\ell} + \ell|\nabla\alpha|^2\,dx, \tag{3.2}$$

where

$$W(\mathbf{e},T) := \frac{1}{2}\mathbf{A}\,(\mathbf{e} - T\alpha_L) : (\mathbf{e} - T\alpha_L)\,.$$

We then introduce a set of rescaling factors and rescaled fields and parameters denoted respectively by X_0 and $\tilde{X}$. We set $x = x_0\tilde{x}$, $u(x) = u_0\tilde{u}(\tilde{x})$, $\alpha(x) = \tilde{\alpha}(\tilde{x})$, $T(x) = T_0\tilde{T}(\tilde{x})$, $\mathbf{A} = A_0\tilde{\mathbf{A}}$, and $\alpha_L = \alpha_0\tilde{\alpha}_L$. For consistency reasons, we set $\ell = \ell_0\tilde{\ell}$. Energy (3.2) can then be re-written as

$$\mathcal{E}_\ell(u,\alpha,T) = \mathbf{A}_0 u_0^2 x_0^{n-1}\int_{\tilde{\Omega}} \frac{1}{2}\tilde{\mathrm{A}}\left(\tilde{\mathbf{e}}(\tilde{u}) - \frac{\alpha_0 x_0 T_0}{u_0}\tilde{\alpha}_L\tilde{T}\right) : \left(\tilde{\mathbf{e}}(\tilde{u}) - \frac{\alpha_0 x_0 T_0}{u_0}\tilde{\alpha}_L\tilde{T}\right)\,d\tilde{x}$$
$$+ G_0 x_0^{n-1}\frac{\tilde{G}}{4c_w}\int_{\tilde{\Omega}} \frac{w(\tilde{\alpha})}{\tilde{\ell}} + \tilde{\ell}\nabla|\tilde{\alpha}|^2\,d\tilde{x}. \tag{3.3}$$

We then pick all scaling coefficients in (3.3) so that all fields are of order 1, *i.e.* $x_0 = \mathcal{O}(\mathrm{diam}(\Omega)$, $T_0 = \Delta T$, $A_0 = E$ (assuming isotropic Hooke's law), $G_0 = G_c$, $u_0 = \sqrt{\frac{G_c x_0}{E}}$. Finally, we pick $\alpha_0 = \frac{u_0}{x_0\Delta T}$, so that the only parameter left is $\tilde{\alpha}_L = \frac{\alpha x_0 \Delta T}{u_0} = \alpha\Delta T\sqrt{\frac{E x_0}{G_c}}$, and the energy becomes:

$$\mathcal{E}_\ell(u,\alpha,T) = G_c x_0^{n-1}\left[\int_{\tilde{\Omega}} \frac{1}{2}\tilde{\mathrm{A}}\left(\tilde{\mathbf{e}}(\tilde{u}) - \tilde{\alpha_L}\tilde{T}\right) : \left(\tilde{\mathbf{e}}(\tilde{u}) - \tilde{\alpha}_L\tilde{T}\right)\,d\tilde{x} + \frac{1}{4c_w}\int_{\tilde{\Omega}} \frac{w(\tilde{\alpha})}{\tilde{\ell}} + \tilde{\ell}\nabla|\tilde{\alpha}|^2\,d\tilde{x}\right]. \tag{3.4}$$

Similarly, using $t_0 = \frac{x_0^2 \rho c_p}{\kappa}$, where the body's thermal conductivity is $K = \kappa \mathrm{I}$, the heat transfer probem becomes

$$\frac{\partial \tilde{T}}{\partial \tilde{t}} = \tilde{\mathrm{div}} \tilde{\nabla} \tilde{T}$$

in Ω, $\tilde{T}(\tilde{x}, \tilde{t} = 0) = 1$, $\tilde{T} = 0$ on $\partial_d \Omega$, and $\frac{\partial \tilde{T}}{\partial n} = 0$ on $\partial_N \Omega$.

Using material properties from [BMMS14], that is $E = 340\,\mathrm{GPa}$, $G_c = 42.47\,\mathrm{J^2\,m^{-1}}$, $\alpha_L = 8 \times 10^{-6}\,\mathrm{K^{-1}}$, $\Delta T = 380\,\mathrm{K}$, and choosing $x_0 = 1\,\mathrm{mm}$ as the reference length, we get $\tilde{\alpha}_L \sim 8.6$. The internal length can be computed from the critical stress in the material, following [PMM11], we have that for the AT1 model, $\sigma_c = \sqrt{\frac{3 G_c E}{8 \ell}}$, so that $\ell = \frac{G_c E}{8 \sigma_c^2}$. From the value $\sigma_c \simeq 342.2\,\mathrm{MPa}$ in [BMMS14], we get that $\tilde{\ell} \simeq 4.63 \times 10^{-3}$, which would require a mesh size of $h = \tilde{\ell}/3 \simeq 1.5 \times 10^{-2}$. When using a coarser mesh, we will capture the parking phenomenon for long cracks but will not be able to predict nucleation time, spacing, and depth correctly.

Journal and data files are located in `Examples/Cooling`

Proposed activities

- Approximate a semi-infinite domain by applying a prescribe temperature along the long edge of the domain, and null flux and roller displacement boundary conditions along all other edges. See how distributed damage arises only above above a critical temperature contrast.
- See how distributed damage evolves into localized damaged areas then into cracks, and how the crack spacing and depth depend on the internal length.
- Highlight the "parking" selection mechanism.
- Re-run on a full or quarter domain.

3.6 Bi-layer

Consider a domain consisting of two layers with a pre-existing crack on the upper layer. Create an inelastic strain by applying a constant temperature field to the layers and applying different elasticity modulii and thermal expansion coefficients to each layer.

Proposed numerical experimentations:

By varying the displacement boundary condition on the lower edge (roller of stress free), and relative mechanical properties in the layers, multiple fracture behaviors can be exhibited: nucleation of periodic crack in the upper later as in the cooling problem, a single crack penetrating in the lower layer before branching, a crack propagating along the interface,... Alternatively, the python script from the surfing example could be used to apply a pre-computed inelastic strain.

3.7 Von Mises Plasticity in 3D

Let consider a domain $\Omega = (-d/2, d/2) \times (-l/2, l/2) \times (0, l), d < l$ in the coordinate (e_1, e_2, e_3) with the associated variables (x_1, x_2, x_3), boundaries conditions applied on the rectangular cuboid are:

- constant traction pre-stresses $\bar{\sigma_2} e_2 \otimes e_2$ applied on surfaces $(*, \pm l/2, *)$
- roller displacement $u.e_3 = 0$ applied on the surface $(*, *, 0)$
- roller displacement $u.e_3 = t$ applied on the surface $(*, *, l)$

The body is subject to Von Mises perfect plasticity criterion ($||\sigma_D|| \leq \sigma_p$) where σ_D is the deviatoric part of the stress. The unique solution of this problem is:

$$\begin{aligned} u(t) = \Big[-\nu(1+\nu)\frac{\bar{\sigma_2}}{E} - \nu t_c - \frac{\bar{\sigma_2} + \bar{\sigma_3}}{2\bar{\sigma_3} - \bar{\sigma_2}}(t - t_c)\Big] x_1 e_1 + \\ \Big[(1-\nu^2)\frac{\bar{\sigma_2}}{E} - \nu t_c + \frac{2\bar{\sigma_2} - \bar{\sigma_3}}{2\bar{\sigma_3} - \bar{\sigma_2}}(t - t_c)\Big] x_2 e_2 + \\ t x_3 e_3 \end{aligned} \tag{3.5}$$

$$p(t) = (t - t_c)\Big[-\frac{\bar{\sigma_2} + \bar{\sigma_3}}{2\bar{\sigma_3} - \bar{\sigma_2}} e_1 \otimes e_1 + \frac{2\bar{\sigma_2} - \bar{\sigma_3}}{2\bar{\sigma_3} - \bar{\sigma_2}} e_2 \otimes e_2 + e_3 \otimes e_3 \Big] \tag{3.6}$$

$$t_c = \frac{1}{2E}\Big((1-2\nu)\bar{\sigma_2} + \sqrt{4\sigma_p^2 - 3\bar{\sigma_2}^2}\Big)$$

$$\bar{\sigma_3} = \frac{1}{2}\Big(\bar{\sigma_2} + \sqrt{4\sigma_p^2 - 3\bar{\sigma_2}^2}\Big)$$

Proposed numerical experimentations:

1. Recover the exact solution of (u, p)

3.8 Damage - Von-Mises Plasticity in 2D

Let consider a 2D rectangular $\Omega = (-L/2, L/2) \times (-H/2, H/2)$ in the coordinate (e_1, e_2) made of metal which are subject to Von Mises Plasticity under plane strain theory. The body is stretched using roller displacement condition at the boundary $u.e_1 = 0$ on $(-L/2, *)$ and $u.e_1 = t$ on $(+L/2, *)$.

Proposed numerical experimentations:

1. By varing the ratio (σ_c/σ_p) capture the transition between brittle fracture and ductile fracture caraterized respectively by a straigth fracture and slant (at 45°) crack path.
2. Do the same but playing with the internal length.What is the influence of the internal length on the behaviors response.
3. Use clamped boundary displacement instead

3.9 Burst experiment in 2D

The idea of burst experiement is to calculate the stress intensity factor of rocks. The experiment consist in keeping the ratio between the injected internal pressure and the confining pressure constant. Then increase the internal pressure until the rock breaks.

Proposed numerical experimentations:

1. Use work controlled with "AT1" to see the crack propagation
2. Use different unilateral contact type and observe the difference of initiation

3.10 Sneddon in 2D

3.10.1 Penny shape crack in plate Sneddon

The Sneddon problem is a penny shape crack of length $(2l_0)$ included in a 2D plate, where a constant pressure is applied on crack tips. Under the assumption of an infinte plate, the solution of the problem is:

$$p(V) = \frac{E'V}{2\pi l_0^2}, \text{ before crack propagation}$$

$$p(V) = \Big(\frac{2G_c^2 E'}{\pi V}\Big)^{1/3}, \text{ during crack propagation}$$

where $E' = E$ in plane stress and $E' = E/(1-\nu^2)$ in plane strain.

Proposed numerical experimentations:

1. Recover the evolution $p(V)$
2. By taking a larger domain and making a loading-unloading-loading cycle try to get better results

3.10.2 Penny shape crack in plate Sneddon

Cruciform pre-cracks in a infinite domain pressurized uniformly.

Proposed numerical experimentations:

1. Observe than only on crack propagates

3.11 Brazilian Test

The brazilian test is a simple and famous test used for rocks and concrete to determine the maximum admissible stress in tension. The idea is to load in compression a cylinder on it's partial lateral surface, by doing that the stress at the enter is in tension. When the stress in tension at the center reaches the critical one an unstable marco crack appears.

Proposed numerical experimentations:

1. Check that a macro cracks appear for $\sigma_{yy} = \sigma_c = \sqrt{\frac{3EG_c}{8(1-\nu^2)\ell}}$ by using unilateral contact masonry.
2. Study the influence of boundary contact surface on the crack inititation.

Appendix A

Appendix

A.1 Plasticity

Let's start to set up the classical variational model for perfect plasticity in the framework of Generalized Standard materials without any damage.

Let (u, p) two state variables defined respectively as the displacement vector and the plastic strain symmetric tensor. The Helmholtz free energy (recoverable internal energy) associated to (u, p) is,

$$W^{free}(u,p) = \frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p)$$

and the maximum dissipation potential associated to the internal dissipation variable p takes the form:

$$\mathrm{H}(\dot{p}) = \sup_{\sigma \in K} \{\sigma : \dot{p}\}$$

The stress denoted $\sigma = -\partial_p W^{free} = \partial_{e(u)} W^{free}$ lies in the admissible set of stress $K := \{\varsigma \in \mathbb{M}_s^{n\times n} | f(\varsigma) \leq 0\}$, the convex set K is closed by the yield surface, $f : \mathbb{M}_s^{n\times n} \to \mathbb{R}$. The flow rule is $\dot{p} \in \partial I_K(\sigma)$, where ∂ is the classical notation in the sens of convex analysis , and $I(\sigma)$ is the indicator function :

$$I_K(\sigma) = \begin{cases} 0 & \text{if } \sigma \in K \\ +\infty & \text{else} \end{cases}$$

Minimizing $\mathcal{E}(u, p)$ consists in choosing the stable state variable trajectory which conserve the energy,

$$\mathcal{E}(u,p) = \int_\Omega \Big[\frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p) + \int_0^t \mathrm{H}(\dot{p})\mathrm{d}\tau\Big]\mathrm{d}x$$

The idea is to follow the classical alternate minimization which is a decreasing energy method. The problem turns out finding a pair (u_t, p_t) at the time t such as

$$(u_t, p_t) \in \underset{u,p}{\operatorname{argmin}} \int_\Omega \Big[\frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p) + \int_0^t \mathrm{H}(\dot{p})\mathrm{d}\tau\Big]\mathrm{d}x$$

$$(u_t, p_t) \in \underset{p}{\operatorname{argmin}} \int_\Omega \int_0^t \mathrm{H}(\dot{p})\mathrm{d}\tau\mathrm{d}x + \underset{u}{\operatorname{argmin}} \int_\Omega \frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p)\mathrm{d}x$$

The minimization problem in u is quadratic and the problem in p can be turned into the minimization of a quadratic functional under the constraint that the stress which has to remains in the elastic domain.

Consider a discretized time interval $0 = t_0 < \cdots < t_{i-1} < t_i < \cdots < t_N = T$. For a given u the minimization in p is:

$$\min_p \frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p) + \int_{t_{i-1}}^{t_i} \mathrm{H}(p-p_{i-1}) + \sum_{k=0}^{i-1} \mathrm{H}(p_k - \dot{p}_{k-1}).$$

The optimality conditions for the minimization problem in p are

$$\begin{aligned} \partial_p \left\{ \frac{1}{2}\mathtt{A}(e(u)-p) : (e(u)-p) + \mathrm{H}(p-p_{i-1}) \right\} \ni 0 \\ -\mathtt{A}(e(u)-p) + \partial_p I_K^*(p-p_{i-1}) \ni 0, \end{aligned} \tag{A.1}$$

where I_K^* is the indicator function in the dual space such as

$$I_K^*(p-p_{i-1}) = \sup_{\sigma\in K}\{\sigma : (p-p_{i-1}) - I_K(\sigma)\} = \mathrm{H}(p-p_{i-1}).$$

Using the Legendre-Fenchel transform, one gets the equivalent formulation

$$\begin{aligned} -\mathtt{A}(e(u)-p) + \partial_p I_K^*(p-p_{i-1}) \ni 0 &\Leftrightarrow -(p-p_{i-1}) + \partial I_K(\mathtt{A}(e(u)-p)) \ni 0 \\ &\Leftrightarrow \mathtt{A}(p-p_{i-1}) - \mathtt{A}\partial I_K(\mathtt{A}(e(u)-p)) \ni 0 \\ &\Leftrightarrow \partial_p\Big(\frac{1}{2}\mathtt{A}(p-p_{i-1}) : (p-p_{i-1}) - I_K(\mathtt{A}(e(u)-p))\Big) \ni 0, \end{aligned} \tag{A.2}$$

and since $I_K(\sigma) = 0$ if $\sigma \in K$, then the minimization problem becomes

$$\min_{\substack{p \\ \mathtt{A}(e(u)-p)\in K}} \frac{1}{2}\mathtt{A}(p-p_{i-1}) : (p-p_{i-1}).$$

A.2 List of all supported YAML options as of 12/13/2018

```yaml
# GLOBAL OPTIONS
verbose:  <1> #Verbosity level
dryrun:   <FALSE> #Dry run in order to validate the options file.
                  # Use in combination with -h to print help or
                  #-verbose 1 to check input deck
```

```
file:
        format:  <EXOSingle> #file format. (choose one of)  EXOSingle EXOSplit
time:
  interpolation: <linear> #interpolation type (choose one of)  linear Vcycle quadratic exo
  min: <0> # first time
  max: <1> # last time
  numstep: <11>: #number of time steps
  skip: <0> #number of time steps skipped
  numCycle: <1> #number of cycles for cyclic loads

# DEFECT MECHANICS GLOBAL OPTIONS
DefMech:
  TimeStepping:
    Type: <QuasiStatic> #Type of defect mechanics Time steping (choose one of)  Null QuasiStatic
  solver:
    Type: <AltMin> #Type of defect mechanics solver (choose one of)  AltMin QuasiNewton1 QuasiNewto
  disp:
    addNullSpace: <TRUE> #handle rigid motion automatically
  displacement:
    Offset: <3> #Position of displacement field in EXO file
  damage:
    Offset: <2> #Position of damage field in EXO file
  boundaryDamage:
    Offset: <0> #Position of boundary damage field in EXO file
  stress:
    Offset: <6> #Position of stress field in EXO file
  temperature:
    Offset: <1> #Position of temperature field in EXO file
  plasticStrain:
    Offset: <0> #Position of the plastic strain field in EXO file
  cumulatedPlasticDissipation:
    Offset: <1> #Position of the Cumulated Plastic Plastic Dissipation field in EXO file
  boundaryDisplacement:
    scaling: <linear> #Boundary displacement scaling (choose one of)  constant linear file null
    Offset: <3> #Position of boundary displacement field in EXO file
  boundaryDamage:
    scaling: <constant> #Boundary damage scaling (choose one of)  constant linear file null
  force:
    scaling: <linear> #Force scaling (choose one of)  constant linear file null
    Offset: <4> #Position of force field in EXO file
  pressureForce:
    scaling: <linear> #Pressure force scaling (choose one of)  constant linear file null
    Offset: <3> #Position of pressure force field in EXO file
  CrackPressure:
    scaling: <linear> #Crack Pressure scaling (choose one of)  constant linear file null
    Offset: <0> #Position of Crack Pressure field in EXO file
```

```
  defmech:
    damage:
      atol: <0.001> #Absolute tolerance on damage error
    maxit: <1000> #Maximum number of alternate minimizations for damage
    pclag: <10> #Interval at which the PC is recomputed during alternate minimization
    irrevThres: <0> #Threshold above which irreversibility is enforced (0 for monotonicity, .99 for
    SOR:
      Omega:  <1> #Alterate Minimization over relaxation factor (>0 for limited, <0 for projected)
    plasticstrain:
      atol: <0.0001> #Absolute tolerance on plastic strain
    InjectedVolume:
      atol: <0.001> #Absolute tolerance on injected volume error
    dampingCoefficient:
      displacement: <0> #Damping coefficient on displacement field (0 for minimization, 1 for semi-
      damage: <0> #Damping coefficient on damage field (0 for minimization, 1 for semi-implicit grad
  BT:
    Type: <Null> #Backtracking type (choose one of)  Null Backward Forward
    Interval: <-1> #Interval at which Backtracking is run in inner loop (0 for outer loop)
    Scope: <-1> #Backtracking scope (0 for unlimited)
    Tol: <0.01>: #Backtracking relative tolerance

# HEAT TRANSFER GLOBAL OPTIONS:
heatxfer:
  timeStepping:
    type: <SteadyState> #Type of heat transfer computation (choose one of)  null SteadyState Transie
  addNullSpace <FALSE> #Add null space to SNES
  initialTemp: <0> #[K] (T): Initial Temperature
  boundaryTemp:
    scaling: <linear> #Boundary temperature scaling (choose one of)  constant linear file null
    Offset: <0> #Position of boundary temperature field in EXO file
  externalTemp:
    scaling: <linear> #External Temperature scaling (choose one of)  constant linear file null
    Offset <2> #Position of external temperature field in EXO file
  flux:
    scaling: <linear> #Heat flux scaling (choose one of)  constant linear file null
    Offset: #Position of heat flux field in EXO file

# CELL SET DEFECT MECHANICS OPTIONS:
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Force: <0 1 2 > #[N.m^(-3) / N.m^(-2) / N.m^(-1)] (f): body / boundary force
  pressureForce: <0> #[N.m^(-2) / N.m^(-1)] (p): boundary pressureforce
  CrackPressure: <0> #[unit-less] internal crack pressure
  damage:
    type: <AT1> #Type of damage law (choose one of)  AT1 AT2 LinSoft KKL AT1Elastic AT2Elastic LinSo
  plasticity:
    type: <None> #Type of plasticity law (choose one of)  None Tresca VonMises VonMisesPlaneTheory C
```

```
  unilateralContact:
    type: <None> #Type of handling of unilateral contact (choose one of)  None HydrostaticDeviatoric
  DisplacementBC: <0 1 2 > #Displacement has Dirichlet boundary Condition (Y/N, yes/no, true/false,
  boundaryDisplacement: <0 1 2 > #[m] (U): Displacement boundary value
  DamageBC: <FALSE> #Damage has Dirichlet boundary Condition (Y/N)
  boundaryDamage: <0> #[unit-less] (alpha): Damage boundary value
  CrackVolumeControlled: <FALSE> #Crack Pressure controlled by the crack volume in this block (Y/N)
  WorkControlled: <FALSE> #Force magnitude controlled by its work in this block (Y/N)

# CELL SET HEAT TRANSFER OPTIONS:
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Flux: <0> #[J.s^(-1).m^(-3) / J.s^(-1).m^(-2)] (f): Internal / applied heat flux
  SurfaceThermalConductivity: <0> #[J.s^(-2).m^(-1).K^(-1)] (H) Surface Thermal Conductivity
  externalTemp: <0> #Reference temperature T [K]
  TempBC: <FALSE> #Temperature has Dirichlet boundary Condition (Y/N)
  boundaryTemp: <0> #Temperature boundary value

# CELL SET MATERIAL PROPERTIES
cs0001: # ID of cell set formatted with 4 digits padded to the left by zeros
  Name: <MEF90Mathium2D> #unused
  Density: <1> #[kg.m^(-2)] (rho) Density
  FractureToughness: <1> #[N.m^(-1)] (G_c) Fracture toughness
  SpecificHeat: <1> #[J.kg^(-1).K^(-1)] (Cp) Specific heat
  ThermalConductivity: <1 2 3 > #[J.m^(-1).s^(-1).K^(-1)] (K) Thermal conductivity
  LinearThermalExpansion: <1 2 3 > #[K^(-1)] (alpha) Linear thermal expansion matrix
  hookeslaw:
    type: <Isotropic> #Type of Hooke's law (choose one of)  Full Isotropic
  hookeslaw:
    YoungsModulus: <1> #[N.m^(-2)] (E) Young's Modulus
    PoissonRatio: <0.3> #[unit-less] (nu) Poisson Modulus
    planeStress: <FALSE> #Use plane stress elasticity (2D only)
    internalLength: <1> #[m] (l) Internal Length
    CoefficientLinSoft: <0> #[] (k) Linear softening coefficient for LinSoft
    yieldStress: <1> #[N.m^(-2)] (sigma_y) stress threshold for plasticity
    residualyieldStress: <0> #[unit-less] (eta) residual yield stress
    DuctileCouplingPower: <2> #[] power of the coupling function b(\alpha) between the damage and th
    CoefficientCapModel0: <-0.3> #C0 in the Yield function: CD || dev(stress) || - C2 tr(stress)^2 +
    CoefficientCapModel1: <0.4> #C1 in the Yield function: CD || dev(stress) || - C2 tr(stress)^2 -
    CoefficientCapModel2: <1> #C2 in the Yield function: CD || dev(stress) || - C2 tr(stress)^2 - C1
    CoefficientCapModelD: <1> #CD in the Yield function: CD || dev(stress) || - C2 tr(stress)^2 - C1
    CoefficientDruckerPrager: <-0.5> #k in the Yield function: || dev(stress) || - k tr(stress) - y1
    CoeffF: <0.3> #[unit-less] (F) coefficient F in the Hill yield criterion
    CoeffG: <0.3> #[unit-less] (G) coefficient G in the Hill yield criterion
    CoeffH: <0.3> #[unit-less] (H) coefficient H in the Hill yield criterion
    CoeffM: <1> #[unit-less] (M) coefficient M in the Hill yield criterion
    CoeffN: <1> #[unit-less] (N) coefficient N in the Hill yield criterion
```

```
  CoeffL: <1> #[unit-less] (L) coefficient L in the Hill yield criterion
  YieldTau0: <1> #[N.m^(-2)] (tau_0) stress threshold in the Hill yield criterion
residualYieldTau0: <0> #[unit-less] residual stress threshold in the Hill yield criterion
phi1: <0> #[radians] Bunge-Euler angle in the Hill yield criterion
phi2: <0> #[radians] Bunge-Euler angle in the Hill yield criterion
Phi: <0> #[radians] Bunge-Euler angle in the Hill yield criterion
delta: <0.0001> #[unit-less] residual in the definition of the porosity, Gurson and Green criteri
cohesiveStiffness: <0> #[N.m^(-4)] (k) cohesive stiffness in Winkler-type models
residualStiffness: <1e-09> #[unit-less] (eta) residual stiffness
isLinearIsotropicHardening: <FALSE> #[bool] Plasticity with Linear Isotropic Hardening
isNoPlCoupling: <FALSE> #[bool] Coupling between damage and plastic dissipation
```

.

Bibliography

[Ale13] R. Alessi. *Variational approach to fracture mechanics with plasticity.* PhD thesis, Università di Roma la Sapienza and École Polytechnique, July 2013.

[AMM09] Hanen Amor, Jean Jacques Marigo, and Corrado Maurini. Regularized formulation of the variational brittle fracture with unilateral contact: Numerical experiments. *Journal of the Mechanics and Physics of Solids*, 57(8):1209–1229, 2009.

[AMMP08] H. Amor, J.-J. Marigo, C. Maurini, and K. Pham. Stability analysis and numerical implementation of non-local damage models via a global variational approach. In *Proceedings of the 8th. World Congress on Computational Mechanics (WCCM8) 5th European Congress on Computational Methods in Applied Sciences and Engineeering (ECCOMAS 2008)*, 2008.

[AMMV17] R. Alessi, J.-J. Marigo, C. Maurini, and S. Vidoli. Coupling damage and plasticity for a phase-field regularisation of brittle, cohesive and ductile fracture: One-dimensional examples. *Int. J. Mech. Sci.*, 2017.

[AT90] L. Ambrosio and V.M. Tortorelli. Approximation of functionals depending on jumps by elliptic functionals via Γ-convergence. *Comm. Pure Appl. Math.*, 43(8):999–1036, 1990.

[AT92] L. Ambrosio and V.M. Tortorelli. On the approximation of free discontinuity problems. *Boll. Un. Mat. Ital. B (7)*, 6(1):105–123, 1992.

[BB+12] S. Balay, J. Brown, , K. Buschelman, V. Eijkhout, W.D. Gropp, D. Kaushik, M.G. Knepley, L. Curfman McInnes, B.F. Smith, and H. Zhang. PETSc users manual. Technical Report ANL-95/11 - Revision 3.3, Argonne National Laboratory, 2012.

[BBB+12] S. Balay, J. Brown, K. Buschelman, W.D. Gropp, D. Kaushik, M.G. Knepley, L. Curfman McInnes, B.F. Smith, and H. Zhang. PETSc Web page, 2012. http://www.mcs.anl.gov/petsc.

[BFM00] B. Bourdin, G.A. Francfort, and J.-J. Marigo. Numerical experiments in revisited brittle fracture. *J. Mech. Phys. Solids*, 48(4):797–826, 2000.

[BFM08] B. Bourdin, G.A. Francfort, and J.-J. Marigo. The variational approach to fracture. *J. Elasticity*, 91(1-3):1–148, 2008.

[BGCMS97] S. Balay, W.D. Gropp, L. Curfman McInnes, and B.F. Smith. Efficient management of parallelism in object oriented numerical software libraries. In E. Arge, A. M. Bruaset, and H. P. Langtangen, editors, *Modern Software Tools in Scientific Computing*, pages 163–202. Birkhäuser Press, 1997.

[BMMS14] B. Bourdin, J.-J. Marigo, C. Maurini, and P. Sicsic. Morphogenesis and propagation of complex cracks induced by thermal shocks. *Phys. Rev. Lett.*, 112(1):014301, 2014.

[Bou07] B. Bourdin. Numerical implementation of a variational formulation of quasi-static brittle fracture. *Interfaces Free Bound.*, 9:411–430, 08 2007.

[CCF18] A. Chambolle, S. Conti, and G. A. Francfort. Approximation of a brittle fracture energy with a constraint of non-interpenetration. *Arch. Rat. Mech. Anal.*, 228(3):867–889, 2018. arXiv:1709.04208.

[FM98] G.A. Francfort and J.-J. Marigo. Revisiting brittle fracture as an energy minimization problem. *J. Mech. Phys. Solids.*, 46(8):1319–1342, 1998.

[Gia05] A. Giacomini. Ambrosio-Tortorelli approximation of quasi-static evolution of brittle fractures. *Calc. Var. Partial Differential Equations*, 22(2):129–172, 2005.

[HHBB14] M.Z. Hossein, C.-J. Hsueh, B. Bourdin, and K. Bhattacharya. Effective toughness of heterogeneous media. Submitted to *J. Mech. Phys. Solids*, 2014.

[KK09] M. Knepley and D. Karpeev. Mesh algorithms for pde with Sieve I: Mesh distribution. *Scientific Programming*, 17(3):215–230, 2009.

[Li16] T. Li. *Gradient Damage Modeling of Dynamic Brittle Fracture*. PhD thesis, Université Paris-Saclay – École Polytechnique, October 2016.

[PAMM11] K. Pham, H. Amor, J.-J. Marigo, and C. Maurini. Gradient damage models and their use to approximate brittle fracture. *International Journal of Damage Mechanics*, 20(4, SI):618–652, 2011.

[PMM11] K. Pham, J.-J. Marigo, and C. Maurini. The issues of the uniqueness and the stability of the homogeneous response in uniaxial tests with gradient damage models. *J. Mech. Phys. Solids*, 59(6):1163 – 1190, 2011.

[SM13] P. Sicsic and J.-J. Marigo. From gradient damage laws to Griffith's theory of crack propagation. *Journal of Elasticity*, 113(1):55–74, 2013.

[Tan17] E. Tanné. *Variational phase-field models from brittle to ductile fracture: nucleation and propagation*. PhD thesis, Université Paris-Saclay, École Polytechnique, December 2017. available from https://pastel.archives-ouvertes.fr/tel-01758354.

[Zeh12] A T Zehnder. *Fracture Mechanics*. Lecture Notes in Applied and Computational Mechanics, No 62. Springer-Verlag, 2012.